

# L2 - 14.2. Building Wazuh Rules

## Task 1 - Introduction

While Wazuh provides extensive built-in detection rules out of the box, organizations must create custom rules to address unique operational risks and security scenarios.

## Task 2 - Decoders

**Data Parsing:** Wazuh can ingest logs from many sources and generate alerts based on their contents. Common formats like JSON, CSV, EVTX, and Syslog are supported out of the box. However, you may encounter logs in an unsupported format, so we need to build a custom **decoder**.

**Understanding Decoders:** To process the raw events and make them available for use in rules and dashboards, Wazuh uses decoders. Decoders are configurable event processors that use regex to extract fields and their values. Let's break down the three blocks the decoder:

| Tag                                   | Purpose                                                                                                                                                  |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>&lt;decoder name="x"&gt;</code> | Name of the decoder. For readability, complex decoders can be split into multiple blocks with the same name, such as we did for thm-web                  |
| <code>&lt;parent&gt;</code>           | Name of the parent decoder. Useful for <u>sibling</u> decoders or when many children decoders (e.g., sysmon/security) inherit one parent (e.g., windows) |
| <code>&lt;prematch&gt;</code>         | Regex pre-filter for optimization. If it matches, decoders start data processing                                                                         |
| <code>&lt;regex&gt;</code>            | Extracts data using regex. Any string inside parentheses is captured (e.g., <code>(\S+)</code> )                                                         |
| <code>&lt;order&gt;</code>            | Maps captured groups to field names (e.g., <code>srcip</code> )                                                                                          |

## Decoding Step by Step

1. The first block verifies the entry starts with an expected pattern:

a. **Regex:** `^\d+/\w+/\d+ \d+:\d+:\d+ WEB`

b. **Matches:** 29/Mar/2025 13:36:36 WEB

c. **Result:** Match is verified, continuing to #2

2. The second block extracts hostname and source IP:

- **Regex:** ^\S+ \S+ WEB (\S+) (\S+)

- **Matches:** 29/Mar/2025 13:36:36 WEB tryhackme.com 203.45.12.88

- **Extracts:** website= tryhackme.com , srcip= 203.45.12.88

3. The third block continues the extraction of other fields:

- **Regex:** "(\w+) (\S+) \S+" (\d+) ...

- **Matches:** " GET /index.php HTTP/1.1" 200 ...

- **Extracts:** method= GET , url= /index.php , status= 200 ...

Decoder Results

```
{  
  "website": "tryhackme.com",  
  "srcip": "203.45.12.88",  
  "method": "GET",  
  "status": "200",  
  "bytes": "1024",  
  "referer": "-",  
  "useragent": "Mozilla/5.0",  
  "url": "/index.php"  
}
```

**Testing Decoders:** In Wazuh, you can test decoders interactively. Open **Server Management > Ruleset Test**. Once in there, paste the raw web logs from the beginning of this task and click **Test**:



You will see that no decoder matched. Let's create a new one in **Server Management > Decoders**. Once in there, search for **local\_decoder.xml** and open it. This is the file for your custom decoders.



Paste the decoder blocks from this task, click **Save**, and then **Reload** to apply the changes. Now, come back to **Ruleset Test**, click **Clear session**, and try again. The logs should be decoded!



All future TryHackMe events will now be decoded into specific fields like **srcip** and **method** so Wazuh rules can process them, enabling you to repeat the previous steps, complete the questions, and advance to the next task.

### Answer the questions below

What decoded (parsed) field contains the value tryhackme.com?

website

```
status: '200'  
url: '/index.php'  
useragent: 'Mozilla/5.0'  
website: 'tryhackme.com'
```

What is the last field extracted by the thm-web decoder?

useragent

## Task 3 - Wazuh Rules

**Wazuh Rules:** Custom Wazuh rules evaluate decoded fields against specific conditions to generate actionable security alerts, such as detecting Sysmon process creation events or identifying suspicious `whoami` discovery commands.

Two Custom Sysmon Rules

```
<rule id="184680" level="0">  
  <match>Microsoft-Windows-Sysmon/Operational: INFORMATION  
(1)</match>  
  <description>Sysmon - Event 1</description>  
  <group>sysmon_event1,</group>  
</rule>  
  
<rule id="184690" level="12">  
  <if_group>sysmon_event1</if_group>  
  <field name="sysmon.image">whoami.exe</field>  
  <description>Sysmon - Whoami Command</description>  
  <mitre><id>T1033</id></mitre>  
  <group>pci_dss_10.6.1,pci_dss_11.4,...</group>  
</rule>
```

A rule block has multiple options. In this case, the options that interest us at this moment are the following:

| Option                                       | Description                                                                                                        |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>rule.id</code>                         | Unique rule ID                                                                                                     |
| <code>rule.level</code>                      | Rule severity from 0 to 15. If the event fits under multiple rules, the one with the highest severity is generated |
| <code>if_group</code> / <code>if_sid</code>  | Specifies that the rule will trigger only when the event has already matched a specific rule group or ID           |
| <code>match</code> / <code>field.name</code> | Specifies the actual rule query by matching full event content or field names with regular expressions             |
| <code>mitre</code> / <code>group</code>      | Contains a list of MITRE techniques, compliance tags, or groups that the rule belongs to                           |

Hierarchical Wazuh rules use low-severity parent rules to capture broad event categories, enabling higher-level child rules to trigger specific security alerts based on finer criteria.

- **0:** Ignored, such rules do not generate alerts in the Wazuh Dashboard
- **1-6:** Regular system events, such as process creation, login, web request
- **7-11:** Notable events or security notifications (e.g., low-priority IDS alerts)
- **12-14:** High-severity events or attack indicators (e.g., suspicious process)
- **15:** Critical alerts that require your immediate attention and response

**Testing Rules:** Continue with the TryHackMe web event source from the previous task, but now use these events: TryHackMe Access Logs Example

```
29/Mar/2025 13:36:36 WEB tryhackme.com 203.45.12.88 "GET /index.php HTTP/1.1" 200 1024 "-" "Mozilla/5.0"
29/Mar/2025 13:37:12 WEB tryhackme.com 91.108.4.201 "POST /admin/login.php HTTP/1.1" 200 1854 "-" "curl/7.68.0"
29/Mar/2025 13:40:18 WEB tryhackme.com 77.83.142.33 "GET /contact.php HTTP/1.1" 200 512 "-" "Mozilla/5.0"
```

Let's say you want to detect a successful login to the admin page. For this, you need two rules.

- The first rule captures logs parsed by the **thm-web** decoder (**level=0**)
- The second detects and alerts on every admin page login (**level=12**)

## Two Custom Web Rules

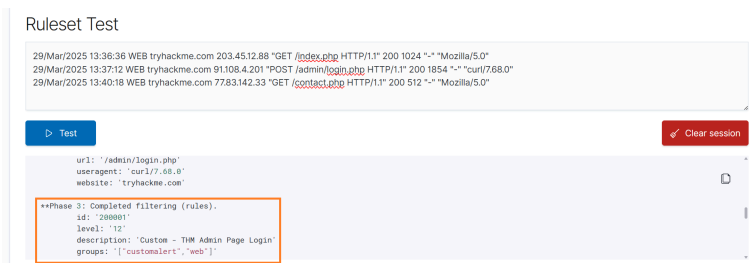
```
<rule id="200000" level="0">
  <decoded_as>thm-web</decoded_as>
  <description>Custom - THM Web Request</description>
  <group>web</group>
</rule>

<rule id="200001" level="12">
  <if_sid>200000</if_sid>
  <status>200</status>
  <url>/admin/login.php</url>
  <field name="method">POST</field>
  <description>Custom - THM Admin Page Login</description>
  <mitre><id>T1078</id></mitre>
  <group>alert,web</group>
</rule>
```

Before testing in Wazuh, note that **status** and **url** filters don't follow the **<field name="x">** format. Wazuh reserves certain field names, such as **status**, **url**, **user**, and **srcip**, so they require dedicated tags, such as **<status>** and **<url>**. Beyond that, the syntax should be familiar to you: all field names come directly from the decoder. Next, upload the rules to the **local\_rules.xml** file via **Server Management > Rules**:



Finally, paste the events into the **Ruleset Test**, clear the session, and verify if any rules matched. In the output, you should see that one out of three events matched the level 12 rule:



Once rules are saved, they will analyze all incoming logs and generate alerts on every match. Check out the **Explore > Discover** page in Wazuh to see the alerts coming from real endpoints.

### Answer the questions below

Which rule description was applied to the third tested web event?

Custom - THM Web Request

Review the prebuilt Wazuh rules in the Rules panel. What is the regex query for the rule that detects PSEXEC?

(?)PSEXESVC\.exe

## Task 4 - Advanced Rules

**CDB Lists (Lookups):** Wazuh CDB (constant database) lists allow custom rules to rapidly compare decoded log fields against pre-compiled lookup files containing threat intelligence indicators or enrichment data to trigger high-priority alerts.

CDB Rule Example

```
<rule id="200002" level="15">
  <if_sid>200000</if_sid>
  <list field="srcip" lookup="address_match_key">etc/lists/
malicious-ioc/malicious-ip</list>
  <description>SOC Alert - THM Web Request From Bad IP</des
cription>
  <mitre><id>T1071</id></mitre>
  <group>alert,web,cti</group>
</rule>
```

The rule above is a classic example of using CDB lists. The rule:

1. Expands the rule with ID 200000 that you applied in the previous task
2. Uses a CDB list located at `[/var/ossec/etc/lists/malicious-ioc/malicious-ip]`
3. Compares the srcip field from the event with every malicious IP within the CDB list
4. If there is a match, it generates a level 15 alert (Critical severity)

**Frequency-Based Rules:** Classic field-matching rules work for independent events, but it is often necessary to alert on suspicious sequences of events, such as brute-force or DoS attacks. You can do it in Wazuh by using the **frequency** and **timeframe** rule properties. For example, the rule below is triggered if there are 3 failed logins within 60 seconds: Frequency-Based Rule

```
<rule id="200003" level="12">
  <if_sid>200000</if_sid>
  <status>401</status>
  <url>/admin/login.php</url>
  <field name="method">POST</field>
  <description>Custom - THM Admin Page Failure</description>
</rule>

<rule id="200004" level="15" frequency="3" timeframe="60">
  <if_matched_sid>200003</if_matched_sid>
  <same_srcip />
  <description>SOC Alert - THM Admin Page Bruteforce</description>
  <mitre><id>T1110</id></mitre>
  <group>alert,web,freq</group>
</rule>
```

The rule with ID 200004 works like this:

1. Checks if the event matches the conditions of rule 200003 ( `<if_matched_sid>` )

2. Groups matched events into buckets by the srcip field ( `<same_srcip />` )
3. Alerts every time the bucket has 3 events within 60 seconds ( `frequency="3" , timeframe="60"` )
4. Comparing the rule above to Splunk's SPL, it is very similar to: `index=web method=POST status=401 url=/admin/login.php| bin _time span=60s| stats count by _time, srcip| where count > 3`

**Rule Pitfalls:** Continuing the frequency-based rules topic, how would Wazuh log 8 login failures from a single IP?

- **Option A:** 8 Admin Page Failure (200003) alerts + 1 Admin Page Bruteforce (200004) alert
- **Option B:** 8 Admin Page Failure (200003) alerts + 2 Admin Page Bruteforce (200004) alerts
- **Option C:** 6 Admin Page Failure (200003) alerts + 2 Admin Page Bruteforce (200004) alerts

The correct answer is **Option C**. Wazuh works pretty simply: it generates at most one alert per event, so if there are 8 login failure events, it will generate eight alerts (or fewer, if some events didn't match any existing rules). For this scenario, if all 8 failures come from a single IP within 60 seconds:

| #1, #2                                                             | #3                                                                                                      | #4, #5                                                             | #6                                                                                                            | #7, #8                                                             |
|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Two regular failure alerts; brute force counter is increased to 2. | Brute force counter hits the threshold, brute force alert is generated. The counter is then reset to 0. | Two regular failure alerts; brute force counter is increased to 2. | Brute force counter hits the threshold again and generates a new alert. The counter is then again reset to 0. | Two regular failure alerts; brute force counter is increased to 2. |

This logic may seem counterintuitive compared to other SIEM solutions and can be confusing when multiple rules "compete" to match the same event. To help reduce alert noise, Wazuh also provides an **ignore** property that suppresses repeated alerts. For example, `<rule id="x" ignore="300">` silences further alerts from the same rule for the next 5 minutes.

**Testing Advanced Rules:** To answer the questions below, edit the **local\_rules.xml** file and add the CDB and frequency-based rules:



```
18 <rule id="200003" level="12">
19   <if_sid>200000</if_sid>
20   <status>0</status>
21   <url>/admin/login.php</url>
22   <field name="method">POST</field>
23   <description>Custom - THM Admin Page Failure</description>
24   <group>alert,web</group>
25 </rule>
26 <rule id="200004" level="15" frequency="3" timeframe="60">
27   <if_matched_sid>200003</if_matched_sid>
28   <name>scip</name>
29   <description>SOC Alert - THM Admin Page BruteForce</description>
30   <mitre><id>T1118</id></mitre>
31   <group>alert,web,free</group>
32 </rule>
33 <rule id="200005" level="15">
34   <if_sid>200001</if_sid>
35   <list field="scip" lookup="address_match_key">etc/lists/malicious-ipc/malicious-ipc</list>
36   <description>SOC Alert - THM Admin Login From Bad IP</description>
37   <mitre><id>T1071</id></mitre>
38   <group>alert,web,cti</group>
39 </rule>
40 </group>
```

Then, open **Ruleset Test** and paste the TryHackMe web logs listed below. Don't forget to click **Clear session** whenever you make changes to the rules:

TryHackMe Access Logs Example

**Answer the questions below**

What MITRE tactic do you see in the Ruleset Test output for the brute-force alert?

Credential Access

Which IP address matched the CDB list and generated a "Bad IP" alert?

158.51.96.38

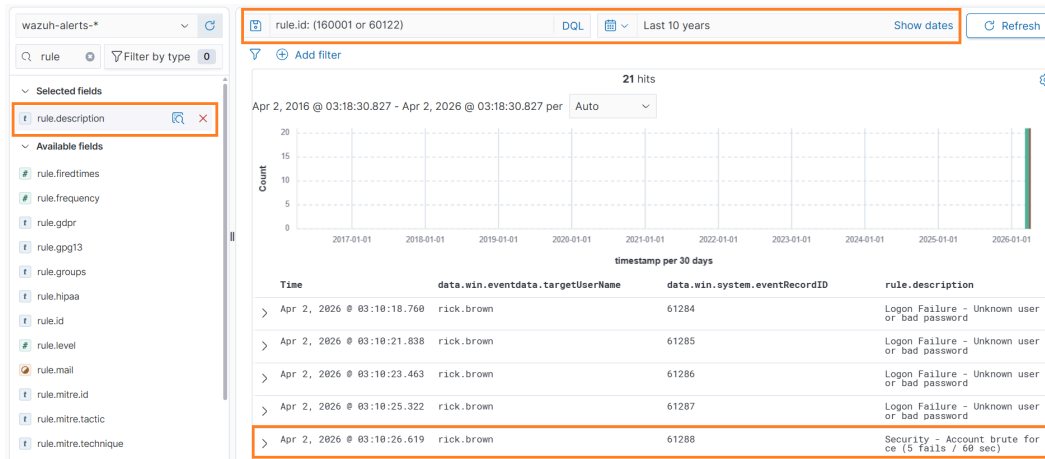
## Task 5 - Mini-Challenge

**SOC Scenario:** Your SOC has received a Wazuh level 15 alert about a brute-force attack on **rick.brown** user to the **SRV-JMP01** server. Open Discover, build the table as shown in the screenshot below, and start the threat investigation! To see the same results as in the screenshot, please follow these steps:

1. Apply this query: `rule.id: (160001 or 60122)` (The query filters for two rules detecting failed logins)
2. Select the time: `Last 10 years` (Or any other time range to include events for April 2026)
3. Build the table:(You can add fields to the table by using the menu on the left)

- `data.win.eventdata.targetUserName`

- `data.win.system.eventRecordID`
- `rule.description`



## Answer the questions below

How many times did the adversary try to log in to rick.brown?

18



Find the definition of the brute force rule in the Wazuh rule files. What is the hidden flag you see at the end of the rule file?

THM{wazuh\_detection\_engineer}

```

17 <!-- Frequency-based rule relying on 4625 event code (60122 SID) -->
18 <rule id="160001" level="12" frequency="5" timeframe="60">
19   <if_matched_sid>60122</if_matched_sid>
20   <same_field>win.eventdata.targetUserName</same_field>
21   <description>Security - Account brute force (5 fails / 60 sec)</description>
22   <mitre><id>T1110</id></mitre>
23 </rule>
24
25 <!-- Your flag: IPM(wazuh_detection_engine) -->
26

```

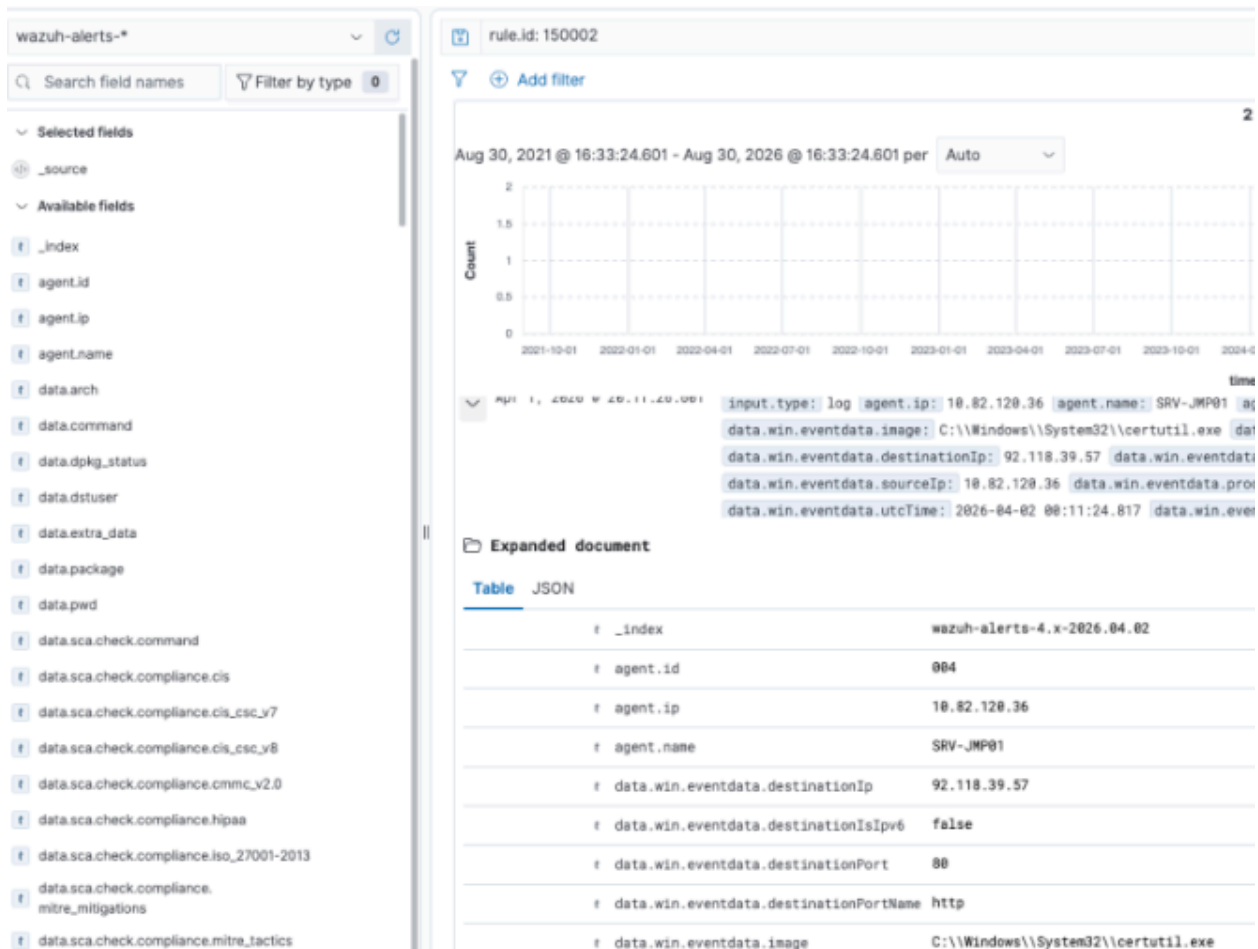
A CDB rule from the same file caught another adversarial action. Which IP address matched the rule and generated Wazuh alerts?

92.118.39.57



Which program initiated the connection to the detected IP address?

certutil.exe



## Task 6 - Fine-Tuning

**Alert Exclusions:** Handling false positives in Wazuh is different from handling them in Splunk or Elastic. Rather than adding NOT filters directly to a rule, exclusions are implemented as child rules with **level="0"**. Let's imagine a scenario below. The rule **100005** detects Auditd file creation events for files containing **malware** or **shell** keywords, and rule **100006** acts as an exclusion for it:

Auditd Rule Example (e.g., `local_rules.xml`)

```

<group name="audit,">
  <rule id="100005" level="12">
    <if_sid>100003</if_sid>
    <field name="audit.file.name">malware|shell</field>
    <description>Audit: $(audit.exe) created a suspicious

```

```

file: $(audit.file.name).</description>
  <group>audit_watch_write,</group>
</rule>

<rule id="100006" level="0">
  <if_sid>100005</if_sid>
  <field name="audit.file.name">shell-checker-thm.sh</field>
  <description>False positive. The script is used by our red team for testing.</description>
  <group>audit_watch_write,</group>
</rule>
</group>

```

### How Wazuh processes exclusions

1. Evaluates the conditions of rule **100005**
2. Checks for child rules and finds **100006**
3. Evaluates the conditions of rule **100006**
  - If matched, the event is mapped to rule **100006**. Since it has **level="0"**, the event is discarded.
  - If not matched, the event is mapped to rule **100005**, and a **level="12"** alert appears in Discover.

**Fine-Tuning:** To modify or tune a built-in rule, avoid editing its original XML file directly, as it may be overwritten during a Wazuh update. Instead, copy the rule into your local rulefile (e.g., **overwrite\_rules.xml**), keep the same rule ID, and use the overwrite property. For example, to extend the rule from the previous section with the detection of **dropper** and **linpeas** keywords, you can use the rule below. Keep in mind that any existing exclusions may also need to be revisited after tuning.

Fine-Tuning Rule Example (e.g., **overwrite\_rules.xml**)

```

<group name="audit,">
  <rule id="100005" level="12" overwrite="yes">

```

```
<field name="audit.file.name">malware|shell|dropper|l  
inpeas</field>  
<description>Audit: $(audit.exe) created a suspicious  
file: $(audit.file.name).</description>  
<group>audit_watch_write,</group>  
</rule>  
</group>
```

Answer the questions below

A Linux process creates a file named shell-checker-thm.sh. What rule ID from the task will be triggered?

100006

A Linux process creates a file named malware.elf. What will be the level of the generated alert?

12

## Task 7 - Conclusion

You should now know how to create custom rules to expand your organization's detection capabilities. You would need to make many rules to cover a wide range of attacks, but each is necessary to improve your security further. As a recap, we learned the following:

- How decoders extract essential data from logs
- How rules use extracted data from decoders
- How rules are triggered and in what order
- How to customize, test, and fine-tune rules